
A First-Order Bundle Method for Smooth Non-convex Multi-objective Optimization

Paul Grigas

University of California, Berkeley
Berkeley, CA 94720
pgrigas@berkeley.edu

Jiahui (Jeffrey) Cheng

University of California, Berkeley
Berkeley, CA 94720
jeffrey_cheng@berkeley.edu

Rongchuan Shi

University of California, Berkeley
Berkeley, CA 94720
rongchuan@berkeley.edu

Hua Ye

University of California, Berkeley
Berkeley, CA 94720
hua01@berkeley.edu

Cheng Zeng

University of California, Berkeley
Berkeley, CA 94720
cheng_zeng@berkeley.edu

Abstract

Multi-objective optimization underlies many machine learning problems, from multi-task learning to LLM safety/alignment. A standard approach scalarizes the objectives via a weight vector in the unit simplex, but in practice the right weights are rarely known, motivating methods that approximate solutions uniformly across all possible weights. Existing approaches either return a single Pareto-stationary point or discretize the simplex and solve each scalarized problem independently, discarding valuable information that could be reused across weights.

We introduce a first-order bundle method that systematically reuses zeroth- and first-order information across weight vectors. The algorithm maintains a bundle of previously computed gradients and function values, from which it constructs a progress criterion certifying solution quality uniformly over the simplex, and adaptively refines the bundle where progress is largest. We establish iteration and oracle complexity bounds under non-convexity — the regime that directly capture common machine learning training settings. We further demonstrate the advantage of our method over the uniform discretization method as the baseline on LLM safety/alignment benchmarks.

1 Introduction

Many problems in machine learning involve the simultaneous optimization of multiple objective functions. In physics-informed neural networks (PINNs) training, multiple loss terms induced by PDE residuals and boundary or initial conditions must be optimized simultaneously. In multi-task learning, a shared model must perform well across several tasks at once, each with its own loss function [R97]. In multi-class classification, the per-class losses define a collection of objectives that must be jointly minimized to achieve good predictive performance across all classes [Bis06]. These settings—and others such as hyperparameter optimization, federated learning, and fairness-aware training—give rise to the *multi-objective optimization* (MOO) problem.

A common approach to MOO is *linear scalarization*: given K objective functions $F_1, \dots, F_K$, one forms the weighted combination $F_\lambda(x) := \sum_{k=1}^K \lambda_k F_k(x)$ for a weight vector λ in the unit simplex Δ_K , and minimizes F_λ using standard first-order methods. For a single, fixed λ , this reduces to a well-understood scalar optimization problem. However, in many settings, such as multi-task learning, the weights need to be assigned manually at prior, and the overall performance depends heavily on the assigned weights. One potential resolution is to find good solutions for *all* weight vectors $\lambda \in \Delta_K$ —that is, to approximate the entire Pareto front. This raises the true challenge of MOO, requiring solving a continuum of optimization problems, one for each λ , which is computationally prohibitive.

In this paper, we introduce a *first-order bundle method* for smooth multi-objective optimization that addresses this gap. The central idea is to maintain a *bundle*—a collection of previously computed zeroth-order and first-order information at a set of iterates—and to construct from this bundle a *progress criterion* that certifies solution quality *uniformly* over all $\lambda \in \Delta_K$.

1.1 Our Methods and Contributions

1. We introduce a general algorithmic framework (Section 4) for bundle-based smooth MOO, which iteratively selects the weight vector λ with the largest progress criterion and refines the bundle at that weight. We present both algorithm iteration complexity and oracle complexity results (Theorem ??).
2. We provide a concrete instantiation of the framework under smoothness and various convexity assumptions (Example 1,2), deriving closed-form expressions for the progress criteria that are efficiently computable from the bundle.
3. We demonstrate the effectiveness of our method on multi-class logistic regression and multi-class cross-entropy loss minimization, as well as multi-task learning benchmarks, showing that the bundle method efficiently traces the Pareto front while reusing gradient information across weight vectors.

1.2 Related Work

Existing approaches address this challenge from different angles. Methods such as the Multiple Gradient Descent Algorithm (MGDA) [Dés12] and its variants [SK18, LLJ⁺21, NSA⁺22] seek a single Pareto-stationary point by combining task gradients, but they do not characterize the full Pareto front. Grid-based scalarization methods [SK18] discretize Δ_K and solve each subproblem independently, but this is wasteful: gradient information computed at one weight vector is discarded when solving for another. Neither approach provides a principled mechanism for *reusing* zeroth-order and first-order information across different scalarization weights.

Notation. For any positive integer m , let $[m]$ denote the set $\{1, \dots, m\}$. Let $\Delta_K := \{\lambda \in \mathbb{R}^K : \sum_{k=1}^K \lambda_k = 1, \lambda \geq 0\}$ be the unit simplex of dimension K . Let $\nabla F(x)$ be the gradient of function F at point $x \in \mathbb{R}^d$. We will make use of a generic given norm $\|\cdot\|$ on $w \in \mathbb{R}^d$.

2 Multi-objective Optimization Preliminaries

We consider multi-objective optimization with K objective functions $F_1(\cdot), \dots, F_K(\cdot)$ where $F_k(\cdot) : \mathbb{R}^d \rightarrow \mathbb{R}$ is L_k -smooth w.r.t. the ℓ_2 -norm and $F_k^* := \inf_{x \in \mathbb{R}^d} F_k(x) > -\infty$ (i.e., is finite). (The norm on x throughout is ℓ_2 .) Note that finiteness of F_k^* is not a strong assumption in many applications that we care about; for example, in machine learning most loss functions are nonnegative. Let $F(x) := (F_1(x), \dots, F_K(x))^\top$ denote a vector of function values at $x \in \mathbb{R}^d$. Notice that we do not make any assumptions on convexity at this point.

Our goal will be to solve the (“scalarized”) multi-objective optimization (MOO) problem:

$$\min_{x \in \mathbb{R}^d} F_\lambda(x) := \lambda^\top F(x) = \sum_{k=1}^K \lambda_k F_k(x), \quad (1)$$

where $\lambda \in \Delta_K$ is a weight vector in the unit simplex in $\mathbb{R}^K$. Note that $F_\lambda(\cdot)$ is L_λ -smooth with $L_\lambda := \sum_{k=1}^K \lambda_k L_k$, and $L_{\max} := \max_k L_k < \infty$. Also, $F_\lambda^* := \inf_{x \in \mathbb{R}^d} F_\lambda(x) \geq \sum_{k=1}^K \lambda_k F_k^* > -\infty$. An approximate solution to problem (1) is referred to as an *approximate solution map* $\hat{x}(\cdot) : \Delta_K \rightarrow \mathbb{R}^d$. To evaluate the quality of an approximate solution map in our potentially non-convex setting, we might use the following measure of worst-case solution map stationarity:

$$\epsilon_{\text{sm-stat}}(\hat{x}(\cdot)) := \sup_{\lambda \in \Delta_K} \|\nabla F_\lambda(\hat{x}(\lambda))\|^2. \quad (2)$$

We might also use the following measure of worst-case function value suboptimality:

$$\epsilon_{\text{sm-gap}}(\hat{x}(\cdot)) := \sup_{\lambda \in \Delta_K} F_\lambda(\hat{x}(\lambda)) - F_\lambda^*.$$

Our goal will be to develop an algorithm, with corresponding complexity analysis, to find an approximate solution map $\hat{x}(\cdot)$ satisfying $\epsilon_{\text{sm-stat}}(\hat{x}(\cdot)) \leq \epsilon$ or possibly $\epsilon_{\text{sm-gap}}(\hat{x}(\cdot)) \leq \epsilon$ for an $\epsilon > 0$.

3 Uniform Discretization — The Baseline Approach

In this section, we describe the uniform discretization method, which serves as the baseline against which we compare Algorithm 2. The idea is to partition the unit simplex Δ_K into a uniform grid, run gradient descent at each grid point, and approximate the solution at any query $\lambda \in \Delta_K$ by rounding to the nearest grid point in L_1 -norm distance.

3.1 Picking the right resolution parameter

Let $r \in \mathbb{Z}_+$ be a grid resolution parameter. Define the uniform simplex grid

$$\mathcal{G}_r := \{\lambda \in \Delta_K : r\lambda \in \mathbb{Z}_+^K\}. \quad (3)$$

That is, $\mathcal{G}_r$ consists of all grid points whose coordinates are multiples of $1/r$. By stars-and-bars, $|\mathcal{G}_r| = \binom{r+K-1}{K-1}$. The goal is to find the correct resolution parameter r for a δ -net of Δ_K .

Proposition 1. For $K \geq 2$,

$$\rho(\mathcal{G}_r) := \sup_{\lambda \in \Delta_K} \min_{\hat{\lambda} \in \mathcal{G}_r} \|\lambda - \hat{\lambda}\|_1 \leq \frac{2\lfloor K^2/4 \rfloor}{K r} \leq \frac{K}{2r}.$$

Proof. Fix $\lambda \in \Delta_K$ and write $y := r\lambda \in \mathbb{R}_+^K$, so that $\sum_{k=1}^K y_k = r$. A grid point $\hat{\lambda} \in \mathcal{G}_r$ corresponds bijectively to $m \in \mathbb{Z}_+^K$ with $\sum_k m_k = r$ via $\hat{\lambda} = m/r$, and $r\|\lambda - \hat{\lambda}\|_1 = \|y - m\|_1$. We construct a feasible m and bound $\|y - m\|_1$.

Step 1 (rounding rule). Let $f_k := y_k - \lfloor y_k \rfloor \in [0, 1)$. Set

$$d := r - \sum_{k=1}^K \lfloor y_k \rfloor = \sum_{k=1}^K f_k,$$

where the second equality uses $\sum_k y_k = r \in \mathbb{Z}$. Since each $f_k < 1$, we have $d \in \{0, 1, \dots, K-1\}$. Choose $S \subseteq [K]$ with $|S| = d$ to index the d largest fractional parts, and define

$$m_k := \begin{cases} \lfloor y_k \rfloor + 1, & k \in S, \\ \lfloor y_k \rfloor, & k \notin S. \end{cases}$$

Then $\sum_k m_k = \sum_k \lfloor y_k \rfloor + d = r$, so m is feasible.

Step 2 (ℓ_1 error). Direct computation:

$$\|y - m\|_1 = \sum_{k \in S} (1 - f_k) + \sum_{k \notin S} f_k = d - \sum_{k \in S} f_k + \left(\sum_k f_k - \sum_{k \in S} f_k \right) = 2d - 2 \sum_{k \in S} f_k,$$

where the last step uses $\sum_k f_k = d$.

Step 3 (lower bound on $\sum_{k \in S} f_k$). The set S collects the d largest of K nonnegative numbers whose average is d/K . The sum of the d largest of any K numbers is at least d times their mean, so

$$\sum_{k \in S} f_k \geq d \cdot \frac{1}{K} \sum_{k=1}^K f_k = \frac{d^2}{K}.$$

Combining with Step 2,

$$\|y - m\|_1 \leq 2d - \frac{2d^2}{K} = \frac{2d(K-d)}{K}. \quad (4)$$

Step 4 (maximizing over d). The right-hand side of (4) is a concave quadratic in d maximized at $d^* = K/2$. Over the admissible integer values $d \in \{0, 1, \dots, K-1\}$ the maximum is attained at $d = \lfloor K/2 \rfloor$, giving

$$\max_{d \in \{0, \dots, K-1\}} \frac{2d(K-d)}{K} = \frac{2 \lfloor K/2 \rfloor \lceil K/2 \rceil}{K} = \frac{2 \lfloor K^2/4 \rfloor}{K}.$$

Step 5 (translating back). Dividing by r ,

$$\min_{\hat{\lambda} \in \mathcal{G}_r} \|\lambda - \hat{\lambda}\|_1 \leq \frac{\|y - m\|_1}{r} \leq \frac{2\lfloor K^2/4 \rfloor}{Kr}.$$

The supremum over $\lambda \in \Delta_K$ gives the first claimed bound; the second follows from $2\lfloor K^2/4 \rfloor/K \leq K/2$.

□

Corollary 3.1 (Resolution for a δ -net). *For any $\delta > 0$, the grid $\mathcal{G}_r$ is an δ -net of Δ_K in the ℓ_1 metric whenever*

$$r \geq \frac{2\lfloor K^2/4 \rfloor}{K\delta},$$

and in particular whenever $r \geq \lceil \frac{K}{2\delta} \rceil$. Equivalently, the smallest such resolution is

$$r^*(\delta) = \left\lceil \frac{K}{2\delta} \right\rceil, \quad |\mathcal{G}_{r^*}| = \binom{r^* + K - 1}{K - 1} = \Theta\left(\frac{1}{(K-1)!} \left(\frac{K}{2\delta}\right)^{K-1}\right).$$

3.2 Uniform error tolerance for the suboptimality gap

The goal of this subsection is to figure out the correct parameter constants, e.g: δ , `grad_norm_tol`, to bound the worst-case suboptimality gap for an approximate solution path $\hat{x}(\cdot)$ defined on a δ -net N of Δ_K :

$$\epsilon_{\text{sm-net}}(\hat{x}(\cdot)) := \sup_{\lambda \in \Delta_K} \min_{\hat{\lambda} \in N} F_\lambda(\hat{x}(\hat{\lambda})) - F_\lambda(x^*(\lambda))$$

for the uniform discretization method.

Proposition 2 (Lipschitz continuity of the optimal solution path). *Define*

- $L_{\max} := \max\{L_i, i \in [k]\}$
- $\mu_{\min} := \min\{\mu_i : i \in [k]\}$
- $G_{\max} := \sup_{\lambda \in \Delta_K} \max_{i \in K} \{\|\nabla F_i(x^*(\lambda))\|\} \leq \max_{i \in [K]} \{L_i \sqrt{\frac{2}{\mu_{\min}}} \sup_{\lambda \in \Delta_K} \sqrt{F_\lambda(0)} + \|\nabla F_i(0)\|\}$

Then, the optimal solution path $\{x^*(\lambda) : \lambda \in \Delta_K\}$ is $\frac{G_{\max}}{\mu_{\min}}$ -Lipschitz in L_1 -norm.

Proof. The proof has two parts. First we show that

$$\|x^*(\hat{\lambda}) - x^*(\lambda)\| \leq \frac{G_{\max}}{\mu_{\min}} \|\lambda - \hat{\lambda}\|_1 \quad \text{for all } \lambda, \hat{\lambda} \in \Delta_K, \quad (5)$$

which is the Lipschitz claim. Then we verify the bound on $G_{\max}$ stated in the proposition.

Step 1: Co-coercivity at the optimum. Since F_λ is μ_λ -strongly convex with unique minimiser $x^*(\lambda)$,

$$\langle \nabla F_\lambda(x^*(\hat{\lambda})), x^*(\hat{\lambda}) - x^*(\lambda) \rangle \geq \mu_\lambda \|x^*(\hat{\lambda}) - x^*(\lambda)\|^2.$$

By Cauchy–Schwarz on the left-hand side and $\mu_\lambda \geq \mu_{\min}$,

$$\mu_{\min} \|x^*(\hat{\lambda}) - x^*(\lambda)\| \leq \|\nabla F_\lambda(x^*(\hat{\lambda}))\|. \quad (6)$$

Step 2: Bounding the right-hand side of (6) by $\|\lambda - \hat{\lambda}\|_1$. Since $\nabla F_{\hat{\lambda}}(x^*(\hat{\lambda})) = 0$ (first-order optimality at $x^*(\hat{\lambda})$),

$$\nabla F_\lambda(x^*(\hat{\lambda})) = \nabla F_\lambda(x^*(\hat{\lambda})) - \nabla F_{\hat{\lambda}}(x^*(\hat{\lambda})) = \sum_{i=1}^K (\lambda_i - \hat{\lambda}_i) \nabla F_i(x^*(\hat{\lambda})).$$

Taking norms and applying the triangle inequality together with the definition of $G_{\max}$ gives

$$\|\nabla F_\lambda(x^*(\hat{\lambda}))\| \leq \sum_{i=1}^K |\lambda_i - \hat{\lambda}_i| \|\nabla F_i(x^*(\hat{\lambda}))\| \leq G_{\max} \|\lambda - \hat{\lambda}\|_1. \quad (7)$$

Combining (6) and (7) yields the Lipschitz inequality (5).

Step 3: Bounding $G_{\max}$. We now establish the explicit upper bound on $G_{\max}$ stated in the proposition. Fix any $\lambda \in \Delta_K$ and any $i \in [K]$. By μ_λ -strong convexity of F_λ together with the nonnegativity convention $F_\lambda \geq 0$ (so that $F_\lambda(x) \geq (\mu_\lambda/2)\|x\|^2$),

$$\frac{\mu_{\min}}{2} \|x^*(\lambda)\|^2 \leq \frac{\mu_\lambda}{2} \|x^*(\lambda)\|^2 \leq F_\lambda(x^*(\lambda)) \leq F_\lambda(0),$$

where the last inequality is the optimality of $x^*(\lambda)$. Hence

$$\|x^*(\lambda)\| \leq \sqrt{\frac{2F_\lambda(0)}{\mu_{\min}}}. \quad (8)$$

Next, by L_i -smoothness of F_i ,

$$\|\nabla F_i(x^*(\lambda))\| = \|\nabla F_i(x^*(\lambda)) - \nabla F_i(0) + \nabla F_i(0)\| \leq L_i \|x^*(\lambda)\| + \|\nabla F_i(0)\|.$$

Substituting (8) and taking the maximum over $i \in [K]$ and the supremum over $\lambda \in \Delta_K$,

$$G_{\max} = \sup_{\lambda \in \Delta_K} \max_{i \in [K]} \|\nabla F_i(x^*(\lambda))\| \leq \max_{i \in [K]} \left\{ L_i \sqrt{\frac{2 \sup_{\lambda \in \Delta_K} F_\lambda(0)}{\mu_{\min}}} + \|\nabla F_i(0)\| \right\},$$

which is the bound claimed in the proposition. $\square$

Theorem 1. *Adopting the notation in Proposition 2, for a uniform error tolerance ϵ , the worst-case suboptimality gap for an approximate solution path $\hat{x}(\cdot)$ defined on a δ -net N of Δ_K is upper bounded by ϵ if*

- *Condition 1:* $\delta \leq \frac{\mu_{\min}}{G_{\max}} \sqrt{\frac{\epsilon}{2L_{\max}}}$.
- *Condition 2:* *grad_norm_tol* for Nesterov's accelerated gradient (NAG) method that is used to solve $\hat{x}(\cdot)$ at each grid point $\hat{\lambda} \in N$ is upper bounded by $\sqrt{\frac{\epsilon}{2L_{\max}}} \mu_{\hat{\lambda}}$.

Proof. Given any $\lambda \in \Delta_K$, let $\hat{\lambda} \in N$ be the closest grid point to λ in L_1 -norm distance.

Define $L_{\max} := \max\{L_i, i \in [K]\}$. Since F_λ is L_λ -smooth, it is also $L_{\max}$ -smooth. By definition,

$$\begin{aligned} F_\lambda(\hat{x}(\hat{\lambda})) - F_\lambda(x^*(\lambda)) &\leq \langle \nabla F_\lambda(x^*(\lambda)), \hat{x}(\hat{\lambda}) - x^*(\lambda) \rangle + \frac{L_{\max}}{2} \|\hat{x}(\hat{\lambda}) - x^*(\lambda)\|^2 \\ &= \frac{L_{\max}}{2} \|\hat{x}(\hat{\lambda}) - x^*(\lambda)\|^2 \\ &\leq \frac{L_{\max}}{2} (\|\hat{x}(\hat{\lambda}) - x^*(\hat{\lambda})\| + \|x^*(\hat{\lambda}) - x^*(\lambda)\|)^2 \end{aligned}$$

To bound the RHS by ϵ , it suffices to bound both $\|\hat{x}(\hat{\lambda}) - x^*(\hat{\lambda})\|$ and $\|x^*(\hat{\lambda}) - x^*(\lambda)\|$ by $\sqrt{\frac{\epsilon}{2L_{\max}}}$.

For the first term, by $\mu_{\hat{\lambda}}$ -strong convexity, we have

$$F_{\hat{\lambda}}(\hat{x}(\hat{\lambda})) - F_{\hat{\lambda}}(x^*(\hat{\lambda})) \geq \frac{\mu_{\hat{\lambda}}}{2} \|\hat{x}(\hat{\lambda}) - x^*(\hat{\lambda})\|^2$$

and

$$F_{\hat{\lambda}}(x^*(\hat{\lambda})) - F_{\hat{\lambda}}(\hat{x}(\hat{\lambda})) \geq \langle \nabla F_{\hat{\lambda}}(\hat{x}(\hat{\lambda})), x^*(\hat{\lambda}) - \hat{x}(\hat{\lambda}) \rangle + \frac{\mu_{\hat{\lambda}}}{2} \|\hat{x}(\hat{\lambda}) - x^*(\hat{\lambda})\|^2$$

Adding both inequalities together to get

$$\langle \nabla F_{\hat{\lambda}}(\hat{x}(\hat{\lambda})), \hat{x}(\hat{\lambda}) - x^*(\hat{\lambda}) \rangle \geq \mu_{\hat{\lambda}} \|\hat{x}(\hat{\lambda}) - x^*(\hat{\lambda})\|^2$$

By Cauchy-Schwartz, we have

$$\|\hat{x}(\hat{\lambda}) - x^*(\hat{\lambda})\| \leq \frac{\|\nabla F_{\hat{\lambda}}(\hat{x}(\hat{\lambda}))\|}{\mu_{\hat{\lambda}}}$$

which is upper bounded by $\sqrt{\frac{\epsilon}{2L_{\max}}}$ if and only if $\|\nabla F_{\hat{\lambda}}(\hat{x}(\hat{\lambda}))\| < \mu_{\hat{\lambda}} \sqrt{\frac{\epsilon}{2L_{\max}}}$

For the second term, apply Proposition 2 to get

$$\|x^*(\hat{\lambda}) - x^*(\lambda)\| \leq \frac{G_{\max}}{\mu_{\min}} \|\hat{\lambda} - \lambda\|_1 < \sqrt{\frac{\epsilon}{2L_{\max}}} \iff \|\hat{\lambda} - \lambda\|_1 < \frac{\mu_{\min}}{G_{\max}} \sqrt{\frac{\epsilon}{2L_{\max}}}.$$

□

3.3 The Uniform Discretization Algorithm

Algorithm 1 Uniform Discretization Algorithm

- 1: **Parameters:** number of classes K , an array of K objective functions, smoothness constants $L \in \mathbb{R}^K$, initial iterate $x_0 \in \mathbb{R}^d$, error tolerance ϵ , first-order method $\mathcal{M}(\cdot)$.
 - 2: Resolution $r = \arg \min_r \sup_{\lambda \in \Delta_K} \min_{\hat{\lambda} \in \mathcal{G}_r} \|\hat{\lambda} - \lambda\|_1 < \frac{\mu_{\min}}{G_{\max}} \sqrt{\frac{\epsilon}{2L_{\max}}} = \lceil \frac{KG_{\max}}{\mu_{\min}} \sqrt{\frac{L_{\max}}{2\epsilon}} \rceil$
 - 3: Enumerate $\mathcal{G}_r = \{\lambda^{(1)}, \dots, \lambda^{(N)}\}$ in an order such that consecutive grid points satisfy $\|\lambda^{(g+1)} - \lambda^{(g)}\|_1 \leq 2/r$ (e.g., a lexicographic sort), where $N = \binom{r+K-1}{K-1}$.
 - 4: Initialize $\hat{x}^{(0)} \leftarrow x_0$.
 - 5: **for** $g = 1, \dots, N$ **do**
 - 6: $\text{grad_norm_tol} = \sqrt{\frac{\epsilon}{2L_{\max}}} \mu_{\lambda^{(g)}}$
 - 7: $\hat{x}^{(g)} \leftarrow \mathcal{M}(\hat{x}^{(g-1)}, \text{grad_norm_tol}, L_{\lambda^{(g)}})$.
 - 8: **end for**
 - 9: return the table $\{(\lambda^{(g)}, \hat{x}^{(g)})\}_{g=1}^N$.
-

At query time, for any $\lambda \in \Delta_K$ the baseline returns

$$\hat{x}_{\text{base}}(\lambda) := \hat{x}^{(g^*)}, \quad g^* = \arg \min_{g \in [N]} \|\lambda - \lambda^{(g)}\|_1. \quad (9)$$

One design choice in Algorithm 1 merits discussion.

Warm-starting. On the first pass through the grid, each new grid point's gradient descent is initialised from the previous grid point's current iterate, exploiting the continuity of the optimal solution map $\lambda \mapsto x^*(\lambda)$ (which is automatic under strong convexity). After the first pass, each grid point continues from its own current iterate, since each $\hat{x}^{(g)}$ is now closer to $x^*(\lambda^{(g)})$ than its neighbour's iterate would be. The lexicographic enumeration ensures consecutive grid points are ℓ_1 -close, which makes the first-pass warm-starting effective.

4 Algorithmic Framework

We propose an algorithmic framework for solving the MOO problem (1) based on a sequence of bundles of zeroth and first-order information as well as associated progress criteria built from these bundles. The key objects in play are:

1. A bundle of previously computed and available zeroth and first-order information, computed at a given list of points $x_1, \dots, x_m$, denoted by

$$\mathcal{B}_m := \{(x_i, F_1(x_i), \dots, F_K(x_i), \nabla F_1(x_i), \dots, \nabla F_K(x_i))\}_{i=1}^m.$$

2. A mapping $T(\cdot; \mathcal{B}_m) : \Delta_K \rightarrow \mathbb{R}^d$, associated with the bundle $\mathcal{B}_m$, where the output $T(\lambda; \mathcal{B}_m)$ is “easily computable” given the information in the bundle $\mathcal{B}_m$.
3. A “gradient norm” function $\text{GN}(\cdot; \mathcal{B}_m) : \Delta_K \rightarrow \mathbb{R}$ associated with the bundle $\mathcal{B}_m$, e.g:

$$\text{GN}(\lambda; \mathcal{B}_m) = \min_{x_i \in \mathcal{B}_m} \{\|\nabla F_\lambda(x_i)\|^2\}.$$

4. A family of constants $\tilde{L}_\lambda > 0$ for $\lambda \in \Delta_K$.

Example 4.1 (Vanilla gradient descent). *For a fixed $\lambda \in \Delta_K$, define a mapping $T(\cdot; \mathcal{B}_m)$ by*

$$T(\lambda; \mathcal{B}_m) := x_{i^*} - \frac{1}{L_\lambda} \nabla F_\lambda(x_{i^*}), \text{ for } i^* \in \arg \min_{i \in [m]} \left\{ F_\lambda(x_i) - \frac{1}{2L_\lambda} \|\nabla F_\lambda(x_i)\|^2 \right\}, \quad (10)$$

with ties broken according to a least index rule (for example).

Note that we require a family of the above objects, indexed by possibly bundles $\mathcal{B}_m$. We first propose some “desired” conditions on these objects, and then follow with specific examples. In the condition below, for a given bundle $\mathcal{B}_m$ of zeroth and first-order information computed at points $x_1, \dots, x_m$ and a given $\bar{x} \in \mathbb{R}^d$, we denote

$$\mathcal{B}_{m+1} := \mathcal{B}_m \cup (\bar{x}, F_1(\bar{x}), \dots, F_K(\bar{x}), \nabla F_1(\bar{x}), \dots, \nabla F_K(\bar{x}))$$

as the new bundle with additional information computed at $\bar{x}$. *Note that whenever a new point is added to the bundle, function values and gradients are assumed to be calculated at this new point.*

4.1 Meta-algorithm

In this subsection, we present a general algorithm to solve the MOO problem (1) for the gradient norm progress criterion function.

Let’s consider the following algorithm. We use a subroutine to iterate adding points to the bundle at a fixed $\lambda \in \Delta_K$. Specifically, let $\mathcal{B}_0$ be a given initial non-empty bundle, and let $\text{BundleUpdate}_M(\lambda; \mathcal{B}_0)$ denote the bundle that results after applying $T(\lambda; \cdot)$ for M steps starting with $\mathcal{B}_0$. In other words, we add $\bar{x}_1 := T(\lambda; \mathcal{B}_0)$ to create bundle $\mathcal{B}_1$, then $\bar{x}_2 := T(\lambda; \mathcal{B}_1)$ to create bundle $\mathcal{B}_2$, etc. until adding M total points to the bundle resulting in the final bundle $\mathcal{B}_M = \text{BundleUpdate}_M(\lambda; \mathcal{B}_0)$.

The adaptive algorithm under non-convexity became

Algorithm 2 Adaptive Algorithm for non-convex function

Input: Number of classes K ; max outer iterations T ; initial point x_0 ; final accuracy parameter ϵ .
Initialize: Initial bundle $\mathcal{B}_0$ built at x_0 ; Anchor $\mathcal{A}_0 = \{\}$.
At iteration $t = 0, 2, \dots, T - 1$ **do:**
 Compute $\lambda_t \in \Delta_K$ that maximizes $\min_{i \in [m]} \|\nabla F_{\lambda_t}(x_i)\|^2$ and set $\text{GN}_t^* := \text{GN}(\lambda_t; \mathcal{B}_t)$ as the maximum value.
 if $\text{GN}_t^* < \frac{2}{3}\epsilon$ **then**
 return anchor $\mathcal{A}_t$ to build the approximate solution path.
 else
 Set $\mathcal{B}_{t+1} = \text{BundleUpdate}_{M_t}(\lambda_t; \mathcal{B}_t)$ where $M_t := \arg \min_{M_t} \{\text{GN}(\lambda_t; \mathcal{B}_{t+1}) < \frac{\epsilon}{3}\}$
 Construct $\mathcal{A}_{t+1}$ from $\mathcal{A}_t$ and $(\lambda_t, \mathcal{B}_{t+1} \setminus \mathcal{B}_t)$
 end if

4.2 Algorithm complexity analysis

Proposition 3. For a bundle $\mathcal{B}_m$ generated by a sequence of points $\{\bar{x}_i := T(\lambda_i; \mathcal{B}_i) : x_i \in \mathcal{B}_m\}$ and any fixed $\lambda \in \Delta_K$, there exists a universal constant $C_\lambda > 0$ such that

$$\text{GN}(\lambda; \mathcal{B}_m) \leq \frac{C_\lambda L_\lambda}{m}.$$

Proof. For gradient descent on an L_λ -smooth (possibly nonconvex) function, Nesterov's descent analysis in Section 1.2.3 yields

$$\min_{i \in [m]} \|\nabla_\lambda F(x_i)\|^2 \leq \frac{1}{m} \left[\frac{L_\lambda}{\omega} (F_\lambda(x_1) - F_\lambda^*) \right],$$

see [Nes18], eqs. (1.2.22). By picking gradient descent step size $\frac{1}{L_\lambda}$, we have $\omega := \frac{1}{2}$.

Hence,

$$\text{GN}(\lambda; \mathcal{B}_m) = \min_{i \in [m]} \|\nabla F_\lambda(x_i)\|^2 \leq \frac{2L_\lambda[F_\lambda(x_1) - F_\lambda^*]}{m}.$$

Let $C_\lambda > 0$ be such that $2[F_\lambda(x_1) - F_\lambda^*] \leq C_\lambda$. □

Corollary 4.1. For any given initial bundle $\mathcal{B}_0$ and any fixed $\lambda \in \Delta_K$, for some universal constant $C_\lambda > 0$, setting $\mathcal{B}_M = \text{BundleUpdate}_M(\lambda; \mathcal{B}_0)$ for $M = \lceil \frac{C_\lambda L_\lambda}{\epsilon} \rceil$ yields

$$\text{GN}(\lambda; \mathcal{B}_M) < \epsilon.$$

Now, we have the following theorem to analyze Algorithm 2.

Theorem 2. Let $\hat{x}(\cdot)$ be the approximate solution path built from the returned anchor $\mathcal{A}$. For each $\lambda \in \Delta_K$, let C_λ be the universal constant in Corollary 4.1. Define

$$\text{LipGN}(\lambda; \mathcal{B}_m) := \max_{x_i \in \mathcal{B}_m} \{\|J_F(x_i) J_F(x_i)^T\|_{1, \infty}\}; \quad E := \sup_{\lambda \in \Delta_K} C_\lambda L_\lambda.$$

Then for some universal constant $C > 0$, after at most $\left(\frac{C \text{LipGN}}{\epsilon}\right)^{K-1}$ outer iterations and at most $\left(\frac{C \text{LipGN}}{\epsilon}\right)^{K-1} \left(\frac{E}{\epsilon}\right)$ total oracle calls, it holds that $\text{GN}_t^* \leq \epsilon$. In addition, $\epsilon_{\text{sm-stat}}(\hat{x}(\cdot)) \leq \epsilon$.

4.3 Idea from Chebyshev paper

At a first glance, the multi-objective optimization problem in the Chebyshev paper is very different from our linear scalarization form. The problem they are solving is

$$\min_{x \in \mathbb{R}^n} (F_1(x), \dots, F_K(x))$$

where each F_k is differentiable, with the convergence result if the total loss $F(x) := \sum_{k=1}^K F_k(x)$ is β -smooth. The problem that we are interested in is to compute the entire (locally) optimal solution path, which is essentially equivalent to

$$\max_{\lambda \in \Delta_K} \min_{x \in \mathbb{R}^n} \sum_{k=1}^K \lambda_k F_k(x)$$

Hence, one idea is to weight each F_k by λ_k and apply algorithm 4, i.e: the optimization problem we are solving in each inner loop became

$$\min_{x \in \mathbb{R}^n} (\lambda_1 F_1(x), \dots, \lambda_K F_K(x))$$

This motivates the following primal and dual formulations (P_λ/D_λ) that is analogous to (P/D):

$$\max_{\|v\| \leq 1, r} r \text{ s.t. } \langle \hat{g}_k, v \rangle \geq \lambda_k r, k \in [K]. \quad (P_\lambda)$$

$$\min_{\beta \geq 0, \sum_{k=1}^K \beta_k \lambda_k = 1} \left\| \sum_{k=1}^K \beta_k \hat{g}_k(x) \right\|. \quad (D_\lambda)$$

And we have the following Weighted-Chebyshev-Exact-ADAM algorithm that is analogous to Algorithm 4.

Algorithm 3 Weighted-Chebyshev-Exact-ADAM

Input: Preference parameter λ , Initial point x_0 , step sizes $\{\eta_t\}_{t=0}^{T-1}$, Adam parameters $\beta_1, \beta_2, \epsilon_{ad}$, threshold $\delta > 0$

Initialize: Adam moments $M_0 = 0, S_0 = 0$

for iteration $t = 0, \dots, T-1$ **do**

 Compute gradients $\lambda_1 \nabla F_1(x_t), \dots, \lambda_K \nabla F_K(x_t)$ and normalize to $\hat{g}_1(x_t), \dots, \hat{g}_K(x_t)$.

$\beta^* = \arg \min_{\beta \geq 0, \sum_{k=1}^K \lambda_k \beta_k = 1} \left\| \sum_{k=1}^K \beta_k \hat{g}_k(x_t) \right\|$ (closed form for $K \leq 3$, else Frank-Wolfe)

$w_t = \sum_{k=1}^K \beta_k^* \hat{g}_k(x_t)$

if $\|w_t\| \leq \delta$ **then**

 Terminate and return x_t

end if

 Recover the primal direction $v_t = \frac{w_t}{\|w_t\|}$

 Set the final update direction with adaptive scalar: $d_t = (\sum_{k=1}^K \langle \lambda_k \nabla F_k(x_t), v_t \rangle) v_t$

$M_{t+1} = \beta_1 M_t + (1 - \beta_1) d_t; \hat{M}_{t+1} = \frac{M_{t+1}}{1 - \beta_1^{t+1}}$

$S_{t+1} = \beta_2 S_t + (1 - \beta_2)(d_t \odot d_t); \hat{S}_{t+1} = \frac{S_{t+1}}{1 - \beta_2^{t+1}}$

$x_{t+1} = x_t - \eta_t \frac{\hat{M}_{t+1}}{\sqrt{\hat{S}_{t+1} + \epsilon_{ad}}}$

end for

return x_T

And we can use the above algorithm as the inner loop for the following main adaptive algorithm.

Algorithm 4 Chebyshev Adaptive Algorithm

Input: Number of classes K ; InnerTol δ ; Adam parameters $(\eta, \beta_1, \beta_2, \epsilon_{ad})$; max outer iters T ; initial point x_0 .
Initialize: Bundle $\mathcal{B}_0$ built at point x_0 ; Anchor $\mathcal{A}_0 = \{(\text{centroid}(\Delta_K), x_0)\}$
for iteration $t = 0, \dots, T - 1$ **do**
 Compute $\lambda_t \in \Delta_K$ that maximizes $\text{PC}(\lambda; \mathcal{B}_t)$ and set $\text{PC}_t^* := \text{PC}(\lambda_t; \mathcal{B}_t)$ as the maximum.
 $(\cdot, x_{\text{near}}) = \arg \min_{(\lambda, x) \in \mathcal{A}_t} \|\lambda - \lambda_t\|$
 $x_t = \text{Weighted-Chebyshev-Exact-ADAM}(\lambda_t, x_{\text{near}}, (\eta, \beta_1, \beta_2, \epsilon_{ad}), \delta)$
 Construct $\mathcal{B}_{t+1}$ from $\mathcal{B}_t$ and x_t
 Construct $\mathcal{A}_{t+1}$ from $\mathcal{A}_t$ and (λ_t, x_t)
end for

5 Numerical Experiments

We empirically compare our Simple Adaptive Algorithm v2 (Algorithm 2) against the uniform-discretisation baseline (Algorithm 1).

Worst-case error. In both experiments we compute a bundle $\mathcal{B}_m$: for the uniform-discretisation baseline (Algorithm 1), $\mathcal{B}_m$ is the set of *last iterates* obtained by running gradient descent on F_λ at each node λ of the coarse grid of resolution r — a fixed set of $\binom{r+K-1}{K-1}$ points spread uniformly over Δ_K ; for Algorithm 2, $\mathcal{B}_m$ is the set of points the algorithm accumulates, allocated adaptively to the weights where the gradient norm progress criterion is largest.

We then measure first-order *stationarity* of the scalarised objective along the solution map, reporting the worst-case squared gradient norm

$$\epsilon_{\text{sm-stat}}(\hat{x}(\cdot)) := \sup_{\lambda \in \Delta_K} \|\nabla F_\lambda(\hat{x}(\lambda))\|^2 = \sup_{(\lambda, x) \in \mathcal{A}} \|\nabla F_\lambda(x)\|^2, \quad (11)$$

Note that

$$\sup_{(\lambda, x) \in \mathcal{A}} \|\nabla F_\lambda(x)\|^2 \leq \sup_{\lambda \in \Delta_K} \min_{x_i \in \mathcal{B}_m} \|\nabla F_\lambda(x_i)\|^2 = \sup_{\lambda \in \Delta_K} \text{GN}(\lambda; \mathcal{B}_m) \quad (12)$$

A small value of (11) certifies that $\hat{x}(\lambda)$ is approximately stationary for F_λ *uniformly* over the simplex Δ_K .

Two budget axes. We benchmark each method along two complementary axes:

1. **Gradient evaluations.** We count one gradient evaluation per call to a single $\nabla F_k(\cdot)$, so each scalarised gradient-descent step costs K evaluations. This is the natural unit for oracle complexity and matches the accounting in Theorem 3. For applications where the gradient is expensive — for instance, large-scale neural-network training where each ∇F_k is a backpropagation pass — this axis directly reflects wall-clock cost.
2. **CPU time.** Algorithm 3 incurs algorithmic overhead beyond gradient evaluation — specifically, maximising the progress criterion over Δ_K , evaluating $\text{GN}(\lambda; \mathcal{B}_m)$ as the bundle grows, and the T -map itself. CPU time captures these costs faithfully and reflects the practical cost when gradient evaluations are cheap.

Checkpoint cadence. For each method, we record (gradient evaluations, CPU time, worst-case error) tuples at periodic checkpoints, triggered when M gradient evaluations have accumulated since the last checkpoint. In both algorithms, the actual checkpoint occurs at the next natural boundary (a pass-completion for the baseline; an outer iteration for Algorithm 3) after the threshold is reached.

Implementation details of Algorithm 2. For Algorithm 2, we make two implementation choices that materially affect runtime cost:

- **Inner-loop pruning.** Standard inner-loop practice would compute $T(\lambda; \mathcal{B}_m)$ and unconditionally evaluate K gradient oracles at the candidate point. We observed that many of

these evaluations contribute negligibly to the bundle once λ is well-served — i.e., once $\|\nabla F_\lambda(x_{i^*})\|$ is small at the active index. We therefore add only the point of the last iterate corresponding to the minimum gradient norm.

Source code. A reference implementation of Algorithm 2, the uniform-discretisation baseline, all progress-criterion variants, and the experiment harness is available at <https://github.com/MarshmallowJeffrey/First-order-method-for-smooth-MOO>.

5.1 Generic non-convexity

For generic non-convexity, we consider multi-layer perceptron (MLP). For simplicity, we consider 1-hidden-layer MLP. In particular, let $W_1 \in \mathbb{R}^{h \times p}$ be the input-to-hidden weights, $b_1 \in \mathbb{R}^h$ be the hidden biases, $W_2 \in \mathbb{R}^{K \times h}$ be the hidden-to-output weights, $b_2 \in \mathbb{R}^K$ be the output biases. Let σ be the activation function, e.g: identity, ReLU.

The forward pass is

$$z := W_2 \sigma(W_1 x + b_1) + b_2.$$

5.1.1 Synthetic data generation

We sample synthetic data from a linear-softmax planted model, identical to the procedure used in subsection 7.2 for the regularised logistic regression experiment. Fix $K = 3$ classes, feature dimension $p = 10$, and sample size $n = 50$. We draw a ground-truth weight matrix and a feature matrix

$$W_\star \in \mathbb{R}^{K \times p}, \quad (W_\star)_{i\ell} \stackrel{\text{iid}}{\sim} \mathcal{U}[-1, 1], \quad x_j \stackrel{\text{iid}}{\sim} \mathcal{N}(0, I_p), \quad j = 1, \dots, n,$$

and assemble $X = [x_1 \cdots x_n]^\top \in \mathbb{R}^{n \times p}$. Conditional on X , the labels are drawn from the categorical distribution induced by the planted linear softmax,

$$y_j | x_j \sim \text{Categorical}(\text{softmax}(W_\star x_j)), \quad j = 1, \dots, n,$$

implemented numerically by computing the row-stochastic matrix $\Pi = \text{softmax}(X W_\star^\top) \in \mathbb{R}^{n \times K}$ and inverse-CDF-sampling each y_j from the j -th row of Π . The pseudo-random seed is fixed (seed = 7) so that $(X, y, W_\star)$ are reproducible across runs.

This generation strategy gives the multi-objective formulation a meaningful target. The over-parameterised hypothesis class – a one-hidden-layer ReLU network of width $h = 8$ with parameter dimension $d = hp + h + Kh + K = 115$ – can represent the planted conditional distribution exactly, so a global optimum of the multi-objective problem exists in parameter space; the per-class cross-entropy losses $F_1, \dots, F_K$ remain genuinely non-convex because of the bilinear composition $W_2 \sigma(W_1 x + b_1)$, providing a clean non-convex testbed that is well-specified in theory yet still requires the optimiser to navigate ReLU kinks and the $W_2 W_1$ product. Because X is standard Gaussian and $W_\star$ has entries in $[-1, 1]$, the ground-truth logits have coordinate-wise standard deviation $\sqrt{p/3}$, so the softmax outputs are moderately peaked and the realised per-class sample counts $n_i = |\{j : y_j = i\}|$ are close to, but not exactly, n/K ; we enforce $n_i \geq 1$ in the per-class average so that each F_i remains well-defined.

5.1.2 Experiment Plots

The next three figures consider the uniform discretization approach for the case $K = 3, p = 10, n = 20, h = 8$:

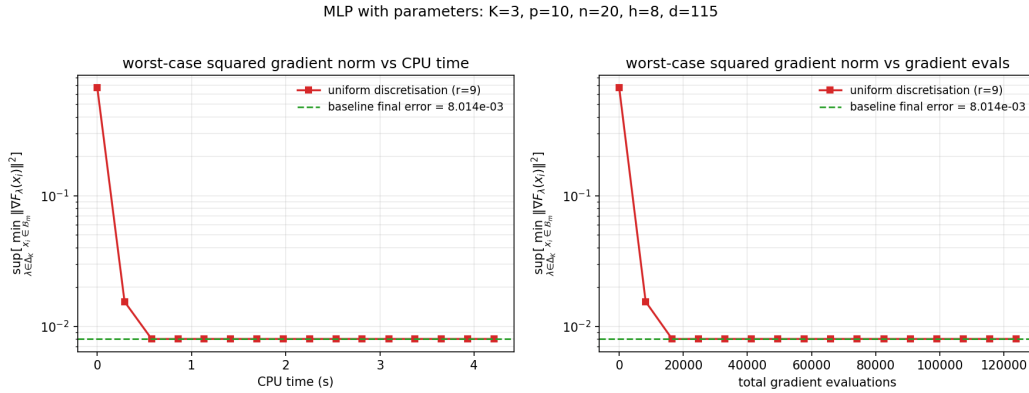


Figure 1: Worst-case squared gradient norm V.S. CPU time and grad evals for error tolerance $1e-2$.

MLP with parameters: K=3, p=10, n=20, h=8, d=115

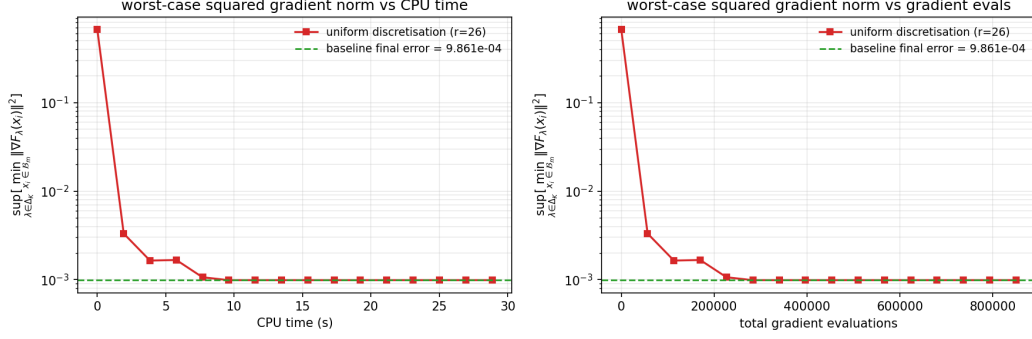


Figure 2: Worst-case squared gradient norm V.S. CPU time and grad evals for error tolerance 1e-3.

MLP with parameters: K=3, p=10, n=20, h=8, d=115

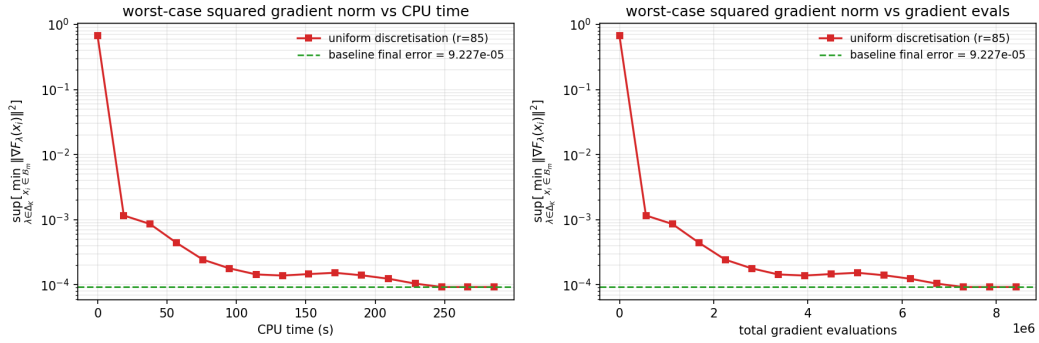


Figure 3: Worst-case squared gradient norm V.S. CPU time and grad evals for error tolerance 1e-4.

5.1.3 LLM safety / LLM alignment

We now show that the MOO framework of this paper applies directly to multi-objective alignment of large language models (LLMs), and in particular to *multi-objective direct preference optimization* (MODPO) [?]. This is a strictly more challenging non-convex testbed than the 1-hidden-layer MLP of Section ?? : the parameter vector θ now indexes a transformer policy π_θ , the per-objective loss is composed through a deep network with softmax outputs over a large vocabulary, and the K objectives correspond to genuinely conflicting alignment dimensions (e.g. helpfulness vs. harmlessness).

Problem setup. Let $\pi_\theta(y | x)$ denote an autoregressive language model with parameters $\theta \in \mathbb{R}^d$, decomposed as $\pi_\theta(y | x) = \prod_{t=1}^{|y|} \pi_\theta(y_t | x, y_{<t})$. Fix a frozen reference policy π_{ref} (typically a supervised fine-tuned checkpoint). Let $R_1, \dots, R_K$ denote K scalar reward models, each capturing a distinct alignment dimension (e.g. R_1 = helpfulness, R_2 = harmlessness, R_3 = honesty). The classical RLHF goal of maximising a *single* reward subject to a KL constraint admits a closed-form optimal policy, which is the well-known starting point for DPO [?]:

$$\pi_{i,\beta}^*(y | x) \propto \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} R_i(x, y)\right), \quad (13)$$

where $\beta > 0$ is the KL-regularisation strength. Inverting this relation yields the implicit reward representation

$$r_i(x, y; \theta) := \beta [\log \pi_\theta(y | x) - \log \pi_{\text{ref}}(y | x)], \quad (14)$$

which expresses any reward R_i directly in terms of the policy parameters θ .

Multi-objective DPO loss. Given a preference dataset $\mathcal{D}_i = \{(x^{(j)}, y_w^{(j)}, y_l^{(j)})\}_{j=1}^{n_i}$ for the i -th alignment dimension — where y_w is the preferred (“winning”) response and y_l the dispreferred (“losing”) one under reward R_i — the DPO loss is the negative log-likelihood of the Bradley–Terry preference model under the implicit reward:

$$F_i(\theta) = -\frac{1}{n_i} \sum_{j=1}^{n_i} \log \sigma(\beta \Delta_\theta^{(j)}), \quad (15)$$

where

$$\Delta_\theta^{(j)} := \log \frac{\pi_\theta(y_w^{(j)} | x^{(j)})}{\pi_{\text{ref}}(y_w^{(j)} | x^{(j)})} - \log \frac{\pi_\theta(y_l^{(j)} | x^{(j)})}{\pi_{\text{ref}}(y_l^{(j)} | x^{(j)})}, \quad (16)$$

and $\sigma(u) = 1/(1 + e^{-u})$ is the logistic function. The MOO formulation (??) then becomes

$$\min_{\theta \in \mathbb{R}^d} F_\lambda(\theta) = \sum_{i=1}^K \lambda_i F_i(\theta), \quad \lambda \in \Delta_K, \quad (17)$$

with F_i defined as in (15) for each preference dataset $\mathcal{D}_i$. Each $\lambda \in \Delta_K$ corresponds to a different user preference profile, and the Pareto front $\{(F_1(\hat{\theta}(\lambda)), \dots, F_K(\hat{\theta}(\lambda))) : \lambda \in \Delta_K\}$ is the set of language model behaviours achievable through preference fine-tuning. As shown in [?, Prop. 1], the global minimiser of F_λ recovers the same optimal policy as multi-objective RLHF (MORLHF) with weight vector λ , so the Pareto front of F is the alignment Pareto front of practical interest.

The current practice in the literature is to discretise Δ_K , run DPO independently at each grid point λ_j , and aggregate the resulting checkpoints — exactly the uniform-discretisation baseline of Section ?? . Our Algorithm ?? with the gradient-norm progress criterion (Example ??, Item 3) replaces this with a principled adaptive schedule that reuses gradient information across λ .

Smoothness of F_i . Each F_i is a composition

$$F_i = -\log \sigma \circ (\beta \Delta_\theta), \quad (18)$$

where Δ_θ is itself a difference of log-likelihoods through the transformer. To make this concrete, write the per-token log-probability as a log-sum-exp over the vocabulary V :

$$\log \pi_\theta(y_t | x, y_{<t}) = z_{\theta,t}(y_t) - \log \sum_{v \in V} \exp z_{\theta,t}(v), \quad (19)$$

where $z_{\theta,t} \in \mathbb{R}^{|V|}$ is the output logit vector of the transformer at position t . F_i is therefore smooth (i.e. has a Lipschitz gradient) on any bounded set of parameters $\Theta \subset \mathbb{R}^d$, because (i) the softmax–logsumexp head is C^∞ with bounded Hessian on bounded logits, (ii) σ' and $(\log \sigma)''$ are bounded by 1/4 and 1 respectively, and (iii) the transformer’s z_θ has a Lipschitz Jacobian on bounded θ . Concretely, for any θ_1, θ_2 in a sublevel set of F_i ,

$$\|\nabla F_i(\theta_1) - \nabla F_i(\theta_2)\| \leq L_i \|\theta_1 - \theta_2\|, \quad (20)$$

with smoothness constant

$$L_i = \mathcal{O}\left(\beta^2 B_z^2 L_{\text{net}} + \beta L_{\text{net}}^{(2)}\right), \quad (21)$$

where $B_z = \sup_{\theta \in \Theta} \|z_\theta\|_\infty$ is an upper bound on the logit magnitudes, L_{net} is a Lipschitz constant of the network Jacobian, and $L_{\text{net}}^{(2)}$ bounds the Hessian of z_θ in θ (i.e. the network’s Jacobian-Lipschitz constant). The β^2 term arises from the implicit reward $r_i = \beta \cdot (\log \pi_\theta - \log \pi_{\text{ref}})$ entering F_i quadratically through σ'' ; the β term comes from the linear coupling between σ' and the logit Jacobian. F_i is non-convex in θ — it inherits the non-convex bilinear structure of every neural-network composition, just like the MLP loss in Section ?? — but is bounded below by 0 (its infimum), satisfying the assumption $F_i^* > -\infty$ used throughout Section ??.

Estimating L_i in practice. As in Section ??, no closed-form constant is available. We estimate L_i empirically by gradient-difference sampling along the optimisation trajectory: at every K_{est} outer iterations we draw N_{pairs} random pairs $(\theta^{(a)}, \theta^{(b)})$ from the recent bundle B_t and compute

$$\hat{L}_i^{(t)} = \max_{(a,b)} \frac{\|\nabla F_i(\theta^{(a)}) - \nabla F_i(\theta^{(b)})\|}{\|\theta^{(a)} - \theta^{(b)}\| + \varepsilon_{\text{num}}}, \quad (22)$$

with $\varepsilon_{\text{num}} = 10^{-8}$ for numerical stability. Because the iterates of Algorithm ?? stay in a sublevel set of F_λ , the locally-estimated $\hat{L}_i$ upper-bounds the relevant Lipschitz constant for the smoothness oracle used in $T(\lambda; B_m)$. When the network is small enough, one can additionally validate $\hat{L}_i$ by power iteration on the Gauss–Newton approximation $\nabla^2 F_i \approx J_\theta^\top H_{\text{softmax}} J_\theta$, where J_θ is the per-sample logit Jacobian.

Data generation strategies. We propose three complementary strategies, ordered by realism vs. controllability.

(1) Planted synthetic preferences (controlled benchmark). This is the LLM analogue of the planted-model generation used in Section ??, and we recommend it as the primary testbed because it admits a known reference Pareto front. Fix a small ground-truth language model $\pi_\star$ and K linear “reward heads” $\{R_i^\star\}_{i=1}^K$, each $R_i^\star(x, y) = \langle w_i^\star, \phi(x, y) \rangle$ for a fixed embedding ϕ . For each i , sample prompts $x \sim P_x$ and pairs of responses $(y_a, y_b) \sim \pi_\star(\cdot | x) \times \pi_\star(\cdot | x)$, then assign preference labels by the Bradley–Terry model

$$\mathbb{P}(y_a \succ_i y_b | x) = \sigma(R_i^\star(x, y_a) - R_i^\star(x, y_b)). \quad (23)$$

Because the reward heads $\{w_i^\star\}$ are known, the optimal scalarised policy $\pi_{\lambda, \beta}^\star$ is computable in closed form for any $\lambda \in \Delta_K$, so the worst-case suboptimality

$$\sup_{\lambda \in \mathcal{G}_{\text{fine}}} [F_\lambda(\hat{\pi}(\lambda)) - F_\lambda^\star]$$

can be evaluated exactly — exactly matching the protocol of Figure ??.

(2) Standard preference-dataset benchmarks (real preferences). Use established multi-objective preference datasets directly: **PKU-SafeRLHF** [?] ($K = 2$: helpfulness, harmlessness; $\sim 30\text{k}$ pairs), **HH-RLHF** [?] ($K = 2$: helpful, harmless; $\sim 170\text{k}$ pairs), or **UltraFeedback** [?] ($K = 4$: helpfulness, truthfulness, honesty, instruction-following; $\sim 64\text{k}$ pairs). In this regime, no ground-truth Pareto front is available, so the worst-case suboptimality is replaced by *hypervolume* [?] over the reward space and *Pareto-front coverage* against the published frontiers of MODPO and Rewarded Soups [?].

(3) Distilled preferences from a stronger model (semi-synthetic). For K beyond 2–4, or for alignment dimensions not covered by existing datasets (e.g. brevity, factuality, code safety), label preferences using a stronger frozen model as oracle. For each prompt x and pair (y_a, y_b) sampled from π_{ref} , query a frozen oracle model for K independent ratings (r_i^a, r_i^b) and assign the i -th preference label as $\mathbf{1}[r_i^a > r_i^b]$. This recovers the controllability of strategy (1) at the cost of (modest) oracle bias.

Experimental configuration. To keep the experiment within the same compute regime as Sections ??–?? while remaining faithful to the LLM-alignment problem, we suggest the following configuration as a starting point: base model GPT-2 small (124M parameters) or Pythia-160M as π_{ref} ; $K = 2$ alignment dimensions on a 5k-pair subset of PKU-SafeRLHF; $\beta = 0.1$; sequence length capped at 128 tokens; reference Pareto front estimated by running plain DPO at 21 evenly-spaced grid points $\lambda \in \{0, 0.05, \dots, 1.0\}$ for a high-accuracy budget. Algorithm ?? is then compared against the uniform-discretisation baseline at matched total gradient-evaluation budgets, with the same (gradient evals, worst-case suboptimality) axes used in Figures ?? and ??.

6 Conclusions and future directions

In this paper, we have developed a first-order bundle method for solving the smooth multi-objective optimization problem, which has broad applications in machine learning, such as multi-task learning, neural architecture search, hyperparameter optimization, generative modeling, transfer and domain adaptation, recommender systems, reinforcement learning with smooth rewards, federated learning with smooth local loss. Some future directions include extending the setting to non-smooth multi-objective optimization, which has applications in fairness-accuracy tradeoff and robust and adversarial learning.

Acknowledgements

References

- [Bis06] C.M. Bishop. *Pattern recognition and machine learning*, volume 4. Springer New York, 2006.
- [Dés12] Jean-Antoine Désidéri. Multiple-gradient descent algorithm (MGDA) for multiobjective optimization. *Comptes Rendus Mathématique*, 350(5–6):313–318, 2012.
- [LLJ⁺21] Bo Liu, Xingchao Liu, Xiaojie Jin, Peter Stone, and Qiang Liu. Conflict-averse gradient descent for multi-task learning. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- [Nes18] Yurii Nesterov. *Lectures on Convex Optimization*. Springer Publishing Company, Incorporated, 2nd edition, 2018.
- [NSA⁺22] Aviv Navon, Aviv Shamsian, Idan Achituve, Haggai Maron, Kenji Kawaguchi, Gal Chechik, and Ethan Fetaya. Multi-task learning as a bargaining game. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 16428–16446. PMLR, 2022.
- [R97] CARUANA R. Multitask learning : A knowledge-based source of inductive bias. *Machine Learning*, 28:41–75, 1997.
- [SK18] Ozan Sener and Vladlen Koltun. Multi-task learning as multi-objective optimization. 31, 2018.

Appendix

A The convergence and oracle complexity of the adaptive algorithm

A.1 Proof of Theorem 2 and Theorem ??

Proof. Consider an $\frac{\epsilon}{3\text{LipPC}}$ -net of Δ_K w.r.t. the ℓ_1 -norm. That is, a finite set of points $N \subseteq \Delta_K$ such that, for all $\lambda \in \Delta_K$, there exists $\hat{\lambda} \in N$ with

$$\|\lambda - \hat{\lambda}\|_1 \leq \frac{\epsilon}{3\text{LipPC}}.$$

By Assumption ??, we have for all $t \geq 1$ and for all $\lambda \in \Delta_K$, there exists $\hat{\lambda} \in N$ such that

$$\text{PC}(\lambda; \mathcal{B}_t) \leq \text{PC}(\hat{\lambda}; \mathcal{B}_t) + \frac{\epsilon}{3}.$$

Therefore, it suffices to show that $\sup_{\hat{\lambda} \in N} \text{PC}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ at some iteration t .

At any iteration t of Algorithm 2, if $\text{PC} = \text{UB}/\text{GAP}$, applying Corollary ?? with $M_t = \left\lceil \frac{\tilde{L}_{\lambda_t}}{\mu_{\lambda_t}} \log(3\text{PC}_t^*/\epsilon) \right\rceil$ yields:

$$\text{PC}(\lambda_t; \mathcal{B}_{t+1}) \leq \left(1 - \frac{\mu_{\lambda_t}}{\tilde{L}_{\lambda_t}}\right)^{M_t} \cdot \text{PC}_t^* \leq \frac{\epsilon}{3}.$$

Otherwise $\text{PC} = \text{GN}$, let C_{λ_t} be the constant in Corollary 4.1 such that

$$\text{PC}(\lambda_t; \mathcal{B}_{t+1}) < \frac{\epsilon}{3}$$

for $M_t = \left\lceil \frac{3C_{\lambda_t}L_{\lambda_t}}{\epsilon} \right\rceil$.

Now, let $\hat{\lambda}_t \in N$ be the projection onto the net N for λ_t . Then, Assumption ?? yields

$$\text{PC}(\hat{\lambda}_t; \mathcal{B}_{t+1}) \leq \text{PC}(\lambda_t; \mathcal{B}_{t+1}) + \frac{\epsilon}{3} \leq \frac{2\epsilon}{3},$$

by combining with the previous inequality.

Thus, at every iteration t of Algorithm 2, we can examine the list of values $\{\text{PC}(\hat{\lambda}; \mathcal{B}_t) \text{ for } \hat{\lambda} \in N\}$. Note that these values are all monotone decreasing by the global monotonicity property in Assumption ??. At every iteration, we just demonstrated that at least one of these values becomes less than or equal to $\frac{2\epsilon}{3}$ and therefore remains less than or equal to $\frac{2\epsilon}{3}$ for all subsequent iterations. Thus, the total number of iterations before it is guaranteed that $\sup_{\hat{\lambda} \in N} \text{PC}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ is bounded by the size of N . As mentioned before, $\sup_{\hat{\lambda} \in N} \text{PC}(\hat{\lambda}; \mathcal{B}_t) \leq \frac{2\epsilon}{3}$ ensures that $\text{PC}_t^* \leq \epsilon$. Note that the size of N can be bounded by $\left(\frac{C\text{LipPC}}{\epsilon}\right)^{K-1}$. This completes the proof. $\square$